

CI/CD Orchestrator

A Distributed Continuous Integration and Delivery Platform

Assignment 8.3: Applying Agile Methodologies in a Team Project

Presentations and Evaluations

Team Members

Manish Acharya	Coordinator and Worker (Rust), Integration, DevOps
Rohan Dhakal	Dashboard (Svelte 5 / TypeScript)
Nimesh Giri	Command Executor (Python / FastAPI)
Prabhat Poudel	Pipeline Parser and Planner (Python)

July 10, 2026

1. Project Overview

CI/CD Orchestrator is our ongoing minor project. It is a distributed continuous integration and delivery platform in the spirit of GitHub Actions or GitLab CI, built and operated by our four-person team. Pipelines are declared in a versioned YAML file (.orchestrator/actions.yml). A push to our self-hosted Forgejo instance fires a webhook, the coordinator plans the pipeline into stages and jobs, and a pool of worker machines (including our own laptops) executes the jobs and streams logs back to a live dashboard. The platform runs on our own server behind Caddy at ci.manishacharya.name.np and, as of the latest sprint, redeploys itself through its own pipeline on every push.

The system has four components, each owned by one member, that meet at documented REST contracts. This split is what made it practical for everyone to own real code while still working as a single Scrum team:

Component	Technology	Owner	Responsibility
Coordinator + Worker	Rust (axum, sqlx, redis)	Manish Acharya	REST API, run and job planning, ready-job queue, worker registry, artifact transport, deployment
Dashboard	Svelte 5, TypeScript, Vite	Rohan Dhakal	Live UI: overview, repo and run pages, log terminals, design system, mock data mode
Command Executor	Python, FastAPI, SQLite	Nimesh Giri	Runs job commands as subprocesses, per-run workspaces, log capture and live streaming
YAML Parser / Planner	Python, pydantic, pytest	Prabhat Poudel	Pipeline schema validation, dependency resolution, cycle detection, execution planning

As the assignment brief notes, implementation itself is not the graded objective. The sections below describe how we applied Agile principles and practices (backlog management, sprints, communication, visualization, testing, and CI/CD) to coordinate real work across four people and several machines.

2. Team, Roles and Working Agreement

We follow Scrum with two-week sprints. Product Owner and Scrum Master duties are assigned to specific members, but they are kept lightweight because everyone's primary role is developer of their own component.

Member	Scrum Role	Notes
Manish Acharya	Product Owner + Developer	Owns backlog ordering and release decisions; integrates the components; runs the server and deployment.
Rohan Dhakal	Scrum Master + Developer	Facilitates standups, planning and retrospectives; keeps the board honest; owns the dashboard end to end.

Member	Scrum Role	Notes
Nimesh Giri	Developer	Owns the command executor and its API spec (SPEC.md); pairs with Manish on the worker and executor contract.
Prabhat Poudel	Developer	Owns the parser/planner and its test suite; pairs with Manish on the coordinator and planner contract.

Working agreement highlights: work happens on feature branches merged by pull request, with a review from the owner of the affected component; contracts between components are written down before implementation (dashboard/README.md documents the full HTTP contract, SPEC.md the executor API); and anything that will be shown at a defense must work on the projector, not just on a laptop.

3. Product Backlog

The backlog lives as issues on our self-hosted Forgejo, mirrored on the project board (Section 6). It is groomed at sprint planning: the Product Owner orders items by demo value and risk, the whole team writes acceptance criteria together, and estimates use planning poker story points (1, 2, 3, 5, 8). Below is the current backlog snapshot: user stories (US), engineering tasks (TSK) and defects (BUG).

3.1 User Stories

ID	User Story	Priority	Acceptance Criteria	Status
US-01	As a developer, I can describe my build as stages, jobs and dependencies in a versioned YAML file so every run is reproducible.	High	Valid YAML yields an ordered execution plan. Unknown stages, self-dependencies and dependency cycles are rejected with a clear message.	Done
US-02	As a developer, I want every push to Forgejo to trigger the pipeline automatically so CI needs no manual step.	High	Push webhook creates a run within 3 s, carrying commit SHA, author and message. Pushes from unregistered repos are rejected with a helpful error.	Done
US-03	As a team member, I can watch runs, stages, jobs and worker health on a dashboard so I can tell what broke and where in under 5 seconds.	High	Overview, repo and run pages poll live data every 3 s. Status is always glyph plus word plus color, never color alone (accessibility).	Done
US-04	As a developer, I can read the full terminal log of any job, live while it runs, so I can debug failures without SSH-ing into machines.	High	Complete stdout/stderr is kept, no truncation. Running jobs stream output to the browser within about 5 s. Stage view shows the full log of every job.	Done
US-05	As a team, we can attach several physical machines as workers so jobs run in parallel across our laptops.	Medium	Workers register and heartbeat over REST. Jobs are claimed from a shared queue.	Done

ID	User Story	Priority	Acceptance Criteria	Status
			Jobs of a dead worker are requeued automatically.	
US-06	As maintainers, we want the orchestrator to build and deploy itself on push (dogfooding) so releases need no manual SSH session.	Medium	Push to main rebuilds images and swaps containers about 10 s after the run passes. A failed build leaves the old version running.	Done
US-07	As a pipeline author, artifacts produced by one job are available to dependent jobs, even when they run on another machine.	Medium	Declared artifact paths upload to the coordinator when a job passes. Jobs listing them in needs download them before starting.	Done
US-08	As a developer, I can register a repository from the dashboard by pasting its Forgejo URL.	Medium	Repo metadata (branch, languages, contributors, pipelines) is fetched and displayed. Deleting a repo keeps its run history.	Done
US-09	As a developer, I can view the pipeline YAML behind each pipeline directly in the dashboard.	Low	Viewer shows the file fetched from Forgejo for the selected pipeline. Switching pipelines refreshes the view.	Done
US-10	As the team, we want the coordinator API protected by a shared secret so only our machines can join the worker pool.	Medium	Requests without the token are rejected. Workers and webhooks are configured with the secret.	Backlog

3.2 Tasks

ID	Task	Owner	Status
TSK-01	Coordinator REST API (axum): runs, jobs, workers, claim/report, artifacts	Manish	Done
TSK-02	Worker binary: registration, heartbeats, claim loop, executor hand-off	Manish	Done
TSK-03	Postgres + Redis stores: run persistence, ready-job queue, worker registry	Manish	Done
TSK-04	Command executor service: async job runner, SQLite job state, per-job log files, cancellation and timeouts	Nimesh	Done
TSK-05	Per-run workspaces: clone REPO_URL@COMMIT_SHA once per machine; artifact bundle upload/download	Nimesh	Done
TSK-06	Live log streaming: chunked log writes plus 2-second progress POSTs to the coordinator	Nimesh, Manish	Done
TSK-07	Pipeline schema and validation (pydantic): stages, needs, env, artifacts	Prabhat	Done
TSK-08	Execution planner: dependency resolution, cycle detection, ordered job output	Prabhat	Done

ID	Task	Owner	Status
TSK-09	Parser regression suite: valid and invalid pipeline fixtures under pytest	Prabhat	Done
TSK-10	Dashboard pages (Svelte 5): overview, repo detail, run detail with pan/zoom pipeline canvas	Rohan	Done
TSK-11	Mock data mode so the dashboard can be developed and demoed without a backend	Rohan	Done
TSK-12	Split-tone design system: OKLCH palette, status vocabulary, terminal island, WCAG AA	Rohan	Done
TSK-13	Docker Compose stack, Caddy TLS reverse proxy, self-deploy pipeline with worker pinning	Manish	Done
TSK-14	Server-Sent Events for sub-second log delivery (replaces polling)	Nimesh, Rohan	Backlog

3.3 Bugs

ID	Bug	Severity	Resolution / Acceptance	Status
BUG-01	Jobs stayed in running state forever when a worker died mid-job, blocking the whole run.	High	Coordinator reaps workers on missed heartbeats and requeues their jobs; the run continues on surviving workers.	Fixed
BUG-02	Race condition: two job reports arriving together could enqueue the same dependent job twice (duplicate execution).	High	Conditional queued-flag flip in Postgres guarantees exactly-once enqueue; verified under concurrent reports.	Fixed
BUG-03	Parser accepted pipelines with dependency cycles and self-dependencies, hanging the planner.	High	bad_cycle.yml and bad_self_dep.yml fixtures are rejected with explicit errors; pytest regressions added.	Fixed
BUG-04	Hard-coded 300 s job timeout killed long Docker builds, making self-deploy impossible.	Medium	Pipelines can raise a per-job JOB_TIMEOUT; the executor still enforces a hard 1-hour cap.	Fixed
BUG-05	Job logs appeared only after a job finished; a 15-minute build looked frozen.	Medium	Executor streams output as it is produced; the dashboard shows it within about 5 s with a streaming indicator.	Fixed
BUG-06	Dashboard YAML viewer kept showing the previous pipeline file after switching pipelines.	Low	Viewer refetches on every pipeline switch; verified in both mock and live modes.	Fixed

4. Sprint Planning and Execution

We ran two-week sprints starting May 18, 2026. Each sprint opens with a planning meeting (goal, committed items, points), runs with daily standups, and closes with a demo/review and a retrospective. The midterm defense on July 2 doubled as a stakeholder review with our supervisor. Velocity settled between 24 and 27 points.

Sprint 0: Inception (May 18 to May 31)

Goal: agree what we are building and who owns what.

Outcome: each member wrote a project brief (the nimesh/rohan/prabhat brief commits in our history), component boundaries and REST contracts were sketched, the backlog was seeded with US-01 to US-05, and the repository and Forgejo project board were created. No implementation points were counted.

Sprint 1: Walking Skeleton (Jun 1 to Jun 14)**Goal: one thin slice of every component, talking over HTTP.**

Sprint Backlog Item	Points	Owner	Result
Coordinator/worker CLI and REST skeleton (register, claim)	5	Manish	Done
Executor job API v1: submit, status, logs, cancel (TSK-04)	5	Nimesh	Done
Pipeline schema + validator v1 (TSK-07)	5	Prabhat	Done
Dashboard wireframes and static HTML prototype	3	Rohan	Done

Velocity: 18. Review: first job executed end to end from a curl-triggered run. Retrospective: component contracts were agreed too late, which caused rework at the worker and executor boundary. Action: write the contract down before implementing (this became SPEC.md and dashboard/README.md). We also moved standup to evenings when everyone is free.

Sprint 2: Persistence, Planning, Real UI (Jun 15 to Jun 28)**Goal: survive restarts, plan real pipelines, replace the prototype UI.**

Sprint Backlog Item	Points	Owner	Result
Job entity, Postgres persistence, Redis ready queue (TSK-03)	8	Manish	Done
Per-run workspaces + artifact transport (TSK-05, US-07)	5	Nimesh	Done
Planner with cycle detection + pytest suite (TSK-08/09, BUG-03)	5	Prabhat	Done
Svelte 5 rebuild, mock mode, design system (TSK-10/11/12)	8	Rohan	Partially done

Velocity: 24 of 26 committed. Review: pipeline runs persisted across restarts; the dashboard was demoed fully from mock data. Retrospective: we overcommitted UI work and the design system spilled 2 points. Action: adopt a mock-first workflow so dashboard work never blocks on backend readiness. We also hosted the stack on the server (June 22) so everyone tests against the same instance.

Sprint 3: Distributed Execution and Dogfooding (Jun 29 to Jul 12, current)**Goal: run across many machines, survive failures, deploy ourselves.**

Sprint Backlog Item	Points	Owner	Result
End-to-end execution across machines, dead-worker requeue (US-05, BUG-01/02)	8	Manish	Done
Forgejo webhook + repo registration (US-02, US-08)	5	Manish, Rohan	Done
Live log streaming end to end (US-04, BUG-05, TSK-06)	5	Nimesh, Manish	Done
Midterm defense demo: 5-laptop live run (all members)	3	Team	Done

Sprint Backlog Item	Points	Owner	Result
Self-deploy pipeline, JOB_TIMEOUT, worker pinning (US-06, BUG-04)	5	Manish	Done
Pipeline YAML viewer + stage log terminals (US-09, BUG-06)	3	Rohan, Manish	Done

Velocity: 27 of 29 committed (defense preparation absorbed the slack). Review highlight: at the July 2 midterm defense, a live run executed across five physical worker laptops, with one worker deliberately killed mid-run and its jobs requeued. Retrospective: the defense showed that logs arriving only after job completion looks frozen on a projector, so log streaming was pulled forward into this sprint. A deploy also landed on the wrong machine once in the mixed worker pool, so worker pinning was added.

Sprint 4: Hardening (Jul 13 to Jul 26, planned)

Planned scope: API authentication (US-10), Server-Sent Events for sub-second logs (TSK-14), worker onboarding documentation, and final defense preparation.

5. Coordination and Communication

Practice	Cadence / Tool	How we use it
Daily standup	Daily, about 15 min, Discord (async on class-heavy days)	Yesterday / today / blockers. Blockers involving another component get a pairing slot the same evening.
Weekly working session	In person, weekly	Integration testing across laptops, sprint demo dry-runs, board grooming.
Pull requests + review	Forgejo, per merge	Feature branches; review required from the owner of any component the change touches (for example, worker and executor changes are reviewed by both Manish and Nimesh).
Written contracts	Repo markdown docs	dashboard/README.md (full HTTP contract), SPEC.md (executor API), DESIGN.md and PRODUCT.md (design decisions with dates), so decisions are not lost in chat history.
Stakeholder review	Midterm / final defense	Supervisor and panel see a live demo; feedback items become backlog entries (log streaming came directly from defense feedback).
Pairing	Ad hoc	Cross-component seams are built in pairs: worker and executor (Manish, Nimesh), coordinator and planner (Manish, Prabhat), dashboard and API (Rohan, Manish).

6. Agile Visualization Techniques

6.1 Kanban Board

The Forgejo project board mirrors the backlog with five columns. Cards carry ID, points and owner; nothing enters Review without a linked PR. Current snapshot:

Column	Cards (current)
Backlog	US-10 API authentication, TSK-14 SSE log streaming, worker onboarding guide

Column	Cards (current)
To Do (Sprint 4)	Final defense slides, rate-limit webhook endpoint
In Progress	Sprint 4 planning notes
Review	(empty)
Done (Sprint 3)	US-02, US-04, US-05, US-06, US-08, US-09, BUG-01, BUG-02, BUG-04, BUG-05, BUG-06

6.2 Sprint Burndown

Points remaining across Sprint 3 (29 committed). The flat segment around July 2 is defense week; the retrospective concluded this was predictable, and Sprint 4 is planned around the final defense date.

Checkpoint	Jun 29	Jul 2	Jul 5	Jul 8	Jul 10
Points remaining	29	21	18	10	2

6.3 The Dashboard as Information Radiator

Our product doubles as the team's information radiator. The live dashboard (pipeline flow canvas, per-stage log terminals, worker utilization graphs, activity calendar) stays open during working sessions, so nobody has to ask whether main is green. Status uses a fixed vocabulary of glyph, word and color (passed, failed, running, pending) applied identically everywhere, which is itself a visualization discipline: one shared meaning, visible to everyone.

7. Testing and Quality Practices

Practice	Detail
Parser regression suite (Prabhat)	pytest over fixture pipelines: good.yml must plan; bad_cycle.yml, bad_self_dep.yml and bad_stage.yml must be rejected with specific errors. Runs before every parser merge, so BUG-03 cannot quietly come back.
Static analysis, Rust (Manish)	cargo check + clippy on every change; the coordinator and worker share typed request/response structs, so contract drift fails at compile time.
Static analysis, dashboard (Rohan)	svelte-check with strict TypeScript (zero errors policy); the entire HTTP contract is typed in one api.ts module.
Executor API testing (Nimesh)	FastAPI's generated Swagger UI plus curl scripts exercise submit/status/logs/cancel, timeout and cancellation paths; SQLite state is inspected between cases.
Mock-mode UAT (Rohan)	The dashboard ships a simulated coordinator (?mode=mock) with deterministic runs, so UI states (failures, pending stages, log tails) are testable without touching the real pipeline.
Integration testing (team)	scripts/demo-pipeline.sh drives a full run; weekly multi-laptop sessions rehearse the defense demo, including killing a worker mid-run to verify requeue (BUG-01).
Code review	Every PR is reviewed by the affected component owner before merge; the review checklist includes the acceptance criteria of the linked backlog item.

Definition of Done

- Acceptance criteria of the linked US/TSK/BUG demonstrably met.
- cargo check / svelte-check / pytest pass locally and in the pipeline.
- PR reviewed and merged to main by the component owner.
- The pipeline run triggered by the merge is green (including self-deploy).
- Contract docs updated when an interface changed.

8. CI/CD and DevOps Practices

Our project both practices and implements CI/CD: the platform runs its own pipeline (dogfooding). The repository carries `.orchestrator/actions.yml`, and every push to main runs it on our own orchestrator:

Stage	Job	Purpose
build	build-info	Records commit and source metrics; uploads build-info.txt as an artifact.
test	validate-pipeline	Verifies the pipeline definition itself is present and parseable.
package	package	Bundles release.tar.gz from upstream artifacts (cross-machine artifact transport in action).
deploy	self-deploy	Rebuilds all Docker images from the pushed commit and swaps the running containers about 10 s after the run passes. Pinned to the server's worker (WORKER_PIN); a failed build leaves production untouched.

DevOps practices around the pipeline:

- Infrastructure as code: the whole stack (coordinator, executor, worker, Postgres, Redis) is one docker-compose.yml; server-specific settings live in an override file, so the repo stays environment-agnostic.
- Self-hosted Git and webhooks: Forgejo delivers push events to the coordinator; every run is traceable to a commit, author and message.
- Deployment without SSH: the deploy job builds on the server and hands the container swap to a detached helper, so the pipeline never kills itself mid-run; rolling back means reverting the commit and pushing.
- Distributed workers: our personal laptops join the worker pool over the public API and share CI load; heartbeat monitoring requeues jobs from dead machines.
- TLS and routing via Caddy (ci.manishacharya.name.np); uptime monitored with Uptime Kuma alongside our other self-hosted services.
- Observability: live log streaming, per-worker utilization graphs and an activity calendar make every deployment inspectable from the dashboard.

9. Conclusion

Agile practice is what let four people with four separate codebases ship a working distributed system in three sprints. The backlog kept scope honest, written contracts and pull-request review kept the component seams stable, and the midterm defense acted as a real stakeholder review whose feedback (live log streaming in particular) landed in the very next

sprint. Because the platform deploys itself through its own pipeline, every practice described in Section 8 is exercised on every push. The main lesson from our retrospectives is simple: agreeing on interfaces early and keeping status visible to the whole team saves far more time than it costs.